A VISUAL GUIDE · GIT & GITHUB

The GitHub workflow on one page

Where a change lives, the **git command** that moves it, and what to **say to Codex** instead.

HOW TO READ IT

- `model.R` modified — Changed file, not yet committed
- C `c3f9a1e` — Commit: snapshot + message
- main · new-figures — Branch names: labels that move
- HEAD — Where you are: the current branch
- `$ git push` — Type it in Terminal
- SAY *"push"* — ...or say it to Codex

ON YOUR MAC | ON GITHUB.COM

Working directory — your files, as edited
`model.R` modified

Staging area — the next commit
`model.R` staged

Local repository — your history
C `c3f9a1e` Fix sign error

GitHub — shared copy: origin
C `c3f9a1e` Fix sign error

add → commit → push

`$ git add model.R` — SAY *"stage model.R"*
`$ git commit -m "Fix sign error"` — SAY *"commit it: fix sign error"*
`$ git push` — SAY *"push my commits"*

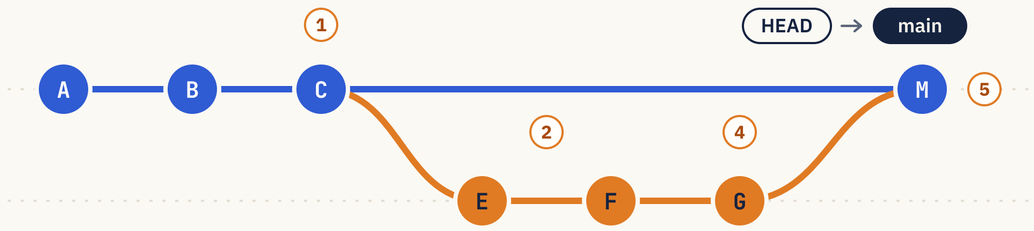
`pull = fetch + merge` `$ git pull` SAY *"sync"*

`save = add + commit + pull + push` `$ git save "Fix sign error"` SAY *"save: fixed the sign error"*

Undo: pick the tool by where the change is

WHERE IT IS	RUN	SAFE AFTER PUSH?
Working directory	`$ git restore model.R` — Throws away your edits to the file. **Warning: they are lost.**	n/a
Staging area	`$ git restore --staged notes.md` — Takes the file out of the next commit; your edits stay.	n/a
Local repository	`$ git uncommit` = `git reset --soft HEAD~1` — Removes the last commit; its changes stay staged.	No
GitHub	`$ git revert --no-edit <hash>` — Adds a new commit that undoes it; then push. History is kept.	Yes

Branches: work on a branch, propose it, merge it



Branch name deleted at step 5; commits E, F, G stay — new-figures

1 **start**	New branch name on C; HEAD moves to it. No file changes.	`$ git switch -c new-figures`	SAY *"start new-figures"*
2 **save**	Commit E, then F, on the branch. main stays at C.	`$ git save "Add fig 3"`	SAY *"save: added fig 3"*
3 **propose**	Upload the branch; open a pull request into main.	`$ git push` `$ gh pr create --fill`	SAY *"propose"*
4 **review**	Asked for a change: commit G, push. The PR updates.	`$ git save "Label y-axis"`	SAY *"save: labelled y-axis"*
5 **finish**	Merge makes M (parents C, G); the name is deleted; you are on main.	`$ gh pr merge --merge --delete-branch` `$ git pull`	SAY *"finish"*

3 PULL REQUEST #12

Add new figures

`new-figures` → `main`

3 commits · +97 −15

Open → Merged

Reviewer: "Label the y-axis."
Fixed in G: each push updates the same pull request.

Conflict: both sides changed the same line

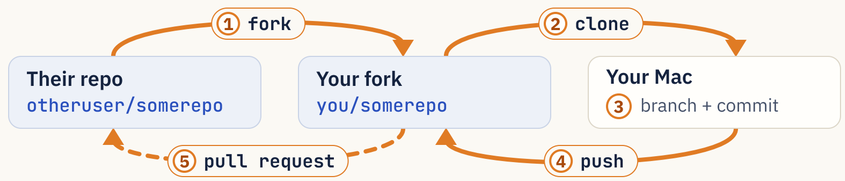
```
<<<<<<< HEAD
alpha <- 0.05        yours
=======
alpha <- 0.10        GitHub's
>>>>>>> 5e6f7a8
```

model.R after `git pull`

`CONFLICT (content): Merge conflict in model.R`

1. Edit the block to the version you want.
2. Delete the three marker lines.
3. `$ git add model.R` `$ git commit --no-edit`
4. `$ git push` — or 3 + 4 in one: `$ git save`

Fork: contribute to someone else's repo



`$ gh repo fork otheruser/somerepo --clone` `$ gh pr create --fill`

The verbs

	status	save	sync	start	propose	finish	undo	new-repo	contribute
SAY	*"status"*	*"save: <what changed>"*	*"sync"*	*"start new-figures"*	*"propose"*	*"finish"*	*"undo <what>"*	*"new-repo sizing-paper"*	*"contribute owner/repo"*
TERMINAL	`git st`	`git save "msg"`	`git sync`	`git start <topic>`	`git propose`	`git finish`	`git uncommit`	–	–
WHAT RUNS	git status, git log	add, commit, pull, push	git pull	switch main, pull, switch -c	push, gh pr create	gh pr merge, switch main, pull, delete branch	restore, reset --soft, revert	git init, commit, gh repo create	gh repo fork --clone